

Cenni alla STL

Cenni alla STL

Panoramica della Standard Template Library del C++ con i contenitori più utili.

Cos'è la STL?

Quando programmi, ti ritrovi spesso ad avere gli stessi problemi: "devo memorizzare una lista di elementi", "devo associare nomi a valori", "devo gestire una coda". Risolverli da zero ogni volta sarebbe faticoso.

La **STL** (Standard Template Library) è una raccolta di "strumenti pronti all'uso" inclusa nel C++. Fornisce strutture dati già pronte (liste, dizionari, code...) e algoritmi già scritti (ordinamento, ricerca...).

Cosa contiene la STL

La STL ha tre componenti principali:

- **Contenitori:** strutture per memorizzare dati (`vector` , `map` , `set` , ...)
- **Algoritmi:** funzioni per operare sui dati (`sort` , `find` , `count` , ...)
- **Iteratori:** oggetti per scorrere i contenitori (trattati nella sezione apposita)

I contenitori principali

`vector` — Lista dinamica

Già trattato nella sezione dedicata. È il contenitore più usato in assoluto.

```
#include <vector>
vector<int> v = {1, 2, 3, 4, 5};
v.push_back(6);
```

map — Dizionario (chiave → valore)

Un `map` associa ogni chiave a un valore. È come un dizionario: cerchi una parola (chiave) e trovi la sua definizione (valore). Le chiavi sono mantenute in ordine alfabetico/numerico.

```
#include <map>
using namespace std;

map<string, int> eta;
eta["Alice"] = 16;
eta["Bob"] = 17;
eta["Carlo"] = 15;

// Leggi il valore associato a una chiave
cout << eta["Alice"] << endl;    // 16

// Scorri tutte le coppie
for (auto& coppia : eta) {
    // .first è la chiave, .second è il valore
    cout << coppia.first << ": " << coppia.second << endl;
}
// Output (in ordine alfabetico):
// Alice: 16
// Bob: 17
// Carlo: 15

// Controlla se una chiave esiste
if (eta.count("Alice") > 0) {
    cout << "Alice è nel registro" << endl;
}

// Rimuovi una coppia
eta.erase("Bob");
```

unordered_map — Dizionario veloce

Come `map`, ma non mantiene l'ordine delle chiavi. In cambio è più veloce:

```
#include <unordered_map>
using namespace std;

unordered_map<string, int> contaParole;
contaParole["ciao"]++; // se la chiave non esiste, viene creata con
valore 0
contaParole["mondo"]++;
contaParole["ciao"]++;

for (auto& p : contaParole) {
    cout << p.first << ": " << p.second << " volte" << endl;
}
```

Usa `map` se ti serve l'ordine, `unordered_map` se ti serve la velocità.

set — Insieme (senza duplicati)

Un `set` memorizza elementi unici, in ordine. Se inserisci un valore già presente, non cambia nulla.

```
#include <set>
using namespace std;

set<int> numeri = {5, 2, 8, 2, 1, 9, 1}; // i duplicati vengono ignorati
// numeri contiene: {1, 2, 5, 8, 9}

numeri.insert(3); // aggiunge 3
numeri.erase(8); // rimuove 8

if (numeri.count(5) > 0) {
    cout << "5 è nel set" << endl;
}

for (int n : numeri) cout << n << " ";
// Output: 1 2 3 5 9
```

list — Lista collegata

Una `list` è efficiente quando aggiungi e rimuovi elementi in mezzo alla lista molto spesso:

```
#include <list>
using namespace std;

list<int> l = {1, 2, 4, 5};
l.push_front(0); // aggiunge all'inizio
l.push_back(6);  // aggiunge alla fine
l.pop_front();   // rimuove il primo

for (int n : l) cout << n << " ";
```

Per la maggior parte dei casi usa `vector`. Usa `list` solo se fai molte inserzioni/rimozioni nel mezzo.

stack — Pila (ultimo entrato, primo uscito)

Una pila (stack) funziona come una pila di piatti: metti sempre sopra, e prendi sempre dall'alto. L'ultimo elemento aggiunto è il primo a uscire (LIFO: Last In, First Out).

```
#include <stack>
using namespace std;

stack<int> pila;
pila.push(1); // metti 1 in cima
pila.push(2); // metti 2 sopra all'1
pila.push(3); // metti 3 sopra al 2

cout << pila.top() << endl; // 3 - elemento in cima
pila.pop();                 // rimuovi l'elemento in cima
cout << pila.top() << endl; // 2
```

queue — Coda (primo entrato, primo uscito)

Una coda (queue) funziona come la cassa al supermercato: il primo ad arrivare è il primo ad essere servito (FIFO: First In, First Out).

```
#include <queue>
using namespace std;

queue<string> coda;
coda.push("Alice");    // Alice arriva per prima
coda.push("Bob");
coda.push("Carlo");

cout << coda.front() << endl; // "Alice" – la prima ad essere servita
coda.pop();              // Alice viene servita e va via
cout << coda.front() << endl; // ora tocca a "Bob"
```

Riepilogo: quale contenitore usare?

Contenitore	Quando usarlo
<code>vector</code>	Uso generale, accesso per posizione
<code>map</code>	Associare chiavi a valori, con ordine
<code>unordered_map</code>	Associare chiavi a valori, più veloce
<code>set</code>	Memorizzare valori unici
<code>list</code>	Molte inserzioni/rimozioni in mezzo
<code>stack</code>	Algoritmi che tornano indietro (backtracking)
<code>queue</code>	Code di elaborazione

Gli algoritmi della STL

Con `#include <algorithm>` hai accesso a molti algoritmi già pronti:

```

#include <algorithm>
#include <vector>
#include <numeric>
using namespace std;

vector<int> v = {5, 2, 8, 1, 9, 3};

// Ordina in ordine crescente
sort(v.begin(), v.end());

// Cerca se un valore è presente (richiede il vector ordinato)
bool trovato = binary_search(v.begin(), v.end(), 5);

// Conta quante volte appare un valore
int quanti = count(v.begin(), v.end(), 3);

// Trova il massimo e il minimo
auto massimo = *max_element(v.begin(), v.end());
auto minimo = *min_element(v.begin(), v.end());
cout << "Max: " << massimo << ", Min: " << minimo << endl;

// Somma tutti gli elementi (da <numeric>)
int somma = accumulate(v.begin(), v.end(), 0);

```